

Homework 3

Ravi Kini

October 31, 2025

Exercise 1

$$H_v = I - 2 \frac{vv^T}{v^T v}$$

Part (a)

Taking the transpose of H_v :

$$\begin{aligned} H_v^T &= I^T - 2 \frac{(vv^T)^T}{v^T v} \\ &= I - 2 \frac{vv^T}{v^T v} = H_v \end{aligned}$$

Therefore H_v is symmetric.

Part (b)

Finding $H_{\alpha v}$:

$$\begin{aligned} H_{\alpha v} &= I - 2 \frac{(\alpha v)(\alpha v)^T}{(\alpha v)^T (\alpha v)} \\ &= I - 2 \frac{\alpha^2 (vv^T)}{\alpha^2 (v^T v)} \\ &= I - 2 \frac{vv^T}{v^T v} = H_v \end{aligned}$$

Therefore H_v is independent of the scaling of α .

Part (c)

Finding H_v^2 :

$$\begin{aligned} H_v^2 &= \left(I - 2 \frac{vv^T}{v^T v} \right)^2 \\ &= I - 4 \frac{vv^T}{v^T v} + 4 \left(\frac{vv^T}{v^T v} \right)^2 \\ &= I - 4 \frac{vv^T}{v^T v} + 4 \frac{v(v^T v)v^T}{(v^T v)^2} \\ &= I - 4 \frac{vv^T}{v^T v} + 4 \frac{vv^T}{v^T v} \\ &= I \end{aligned}$$

Therefore H_v is involutory.

Part (d)

Since H_v is involutory, $H_v^{-1} = H_v$.

Exercise 2

Let $x, y \in \mathbb{R}^n$ such that $x \neq y$ and $\|x\|_2 = \|y\|_2$. Then:

$$\begin{aligned} H_v x &= \left(I - 2 \frac{v v^T}{v^T v} \right) x y = x - 2v \frac{v^T x}{v^T v} \\ 2v \frac{v^T x}{v^T v} &= x - y \\ v &= (x - y) \frac{v^T v}{2v^T x} \end{aligned}$$

Since scaling v should not affect the formula, we choose $v = x - y$.

Exercise 3

$$A = \begin{bmatrix} 1 & 1 & 1 \\ 0 & 1 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

The first column $a_1 = e_1$ already, so we move on to the second column and focus on the 2×2 submatrix, letting $R_1 = A, Q_1 = I$. The second column of A has norm $\sqrt{1^2 + 1^2} = \sqrt{2}$, so we let $v = a_2 - \sqrt{2}e_2 = (1 - \sqrt{2}, 1)^T$. This yields the elimination matrix:

$$\begin{aligned} H_v &= \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} - \frac{2}{4 - 2\sqrt{2}} \begin{bmatrix} 3 - 2\sqrt{2} & 1 - \sqrt{2} \\ 1 - \sqrt{2} & 1 \end{bmatrix} \\ &= \begin{bmatrix} \sqrt{2}/2 & \sqrt{2}/2 \\ \sqrt{2}/2 & -\sqrt{2}/2 \end{bmatrix} \end{aligned}$$

Then, expanding to 3×3 :

$$\begin{aligned} R_2 &= H_v R_1 = \begin{bmatrix} 1 & 1 & 1 \\ 0 & \sqrt{2} & \sqrt{2}/2 \\ 0 & 0 & \sqrt{2}/2 \end{bmatrix} \\ Q_2 &= \begin{bmatrix} 1 & 0 & 0 \\ 0 & \sqrt{2}/2 & \sqrt{2}/2 \\ 0 & \sqrt{2}/2 & -\sqrt{2}/2 \end{bmatrix} \end{aligned}$$

Since R is now upper triangular, we have:

$$A = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \sqrt{2}/2 & \sqrt{2}/2 \\ 0 & \sqrt{2}/2 & -\sqrt{2}/2 \end{bmatrix} \begin{bmatrix} 1 & 1 & 1 \\ 0 & \sqrt{2} & \sqrt{2}/2 \\ 0 & 0 & \sqrt{2}/2 \end{bmatrix}$$

This is the same factorization that is found by using the Gram-Schmidt method.

Exercise 4

If $a \in \mathbb{R}^n$ is treated as a $n \times 1$ matrix A , for the full QR factorization, Q must be a $n \times n$ matrix and R must a $n \times 1$ upper triangular matrix. The columns of Q will be the n vectors found using the Gram-Schmidt method; we define $q_i = e_i - \sum_{j=1}^{i-1} \text{proj}_{q_j}(e_i)$, with $q_0 = a/\|a\|_2$. Let $q'_1, \dots, q'_n$ be the first n nonzero vectors produced by this method. R then has $\|a\|_2$ as its first entry, and all other entries be zero.

$$a = A = \begin{bmatrix} a/\|a\|_2 & q'_2 & \cdots & q'_n \end{bmatrix} \begin{bmatrix} \|a\|_2 \\ 0 \\ \vdots \\ 0 \end{bmatrix}$$

For the reduced QR factorization, we look only at the first entry of R , as all following are zero, and correspondingly, the first column of Q :

$$a = A = \begin{bmatrix} a/\|a\|_2 \end{bmatrix} \begin{bmatrix} \|a\|_2 \end{bmatrix}$$

Exercise 5

$$Q = A(L^T)^{-1}$$

Part

Taking the transpose of Q :

$$\begin{aligned} Q^T &= (A(L^T)^{-1})^{-1} \\ &= L^{-1}A^T \end{aligned}$$

Then:

$$\begin{aligned} QQ^T &= A(L^T)^{-1}L^{-1}A^T \\ &= A(LL^T)^{-1}A^T \\ &= A(A^T A)^{-1}A^T \\ &= AA^{-1}(A^T)^{-1}A^T \\ &= I \end{aligned}$$

Therefore Q is an orthonormal matrix.

Part

If LL^T is the Cholesky factorization of $A^T A$, then in the QR factorization of A , $Q = A(L^T)^{-1}$ and $R = L^T$.

Exercise 6

We seek to minimize:

$$\phi(\alpha, \beta) = \sum_{i=1}^m ((a + \beta t_i) - \sqrt{t_i})^2$$

where $t_i = \frac{1}{4} + \frac{3}{4} \frac{i-1}{m-1}$. We solve the least squares problem where:

$$A = \begin{bmatrix} 1 & t_1 \\ 1 & t_2 \\ \vdots & \vdots \\ 1 & t_m \end{bmatrix}$$
$$x = \begin{bmatrix} \alpha \\ \beta \end{bmatrix}$$
$$b = \begin{bmatrix} \sqrt{t_1} \\ \sqrt{t_2} \\ \vdots \\ \sqrt{t_m} \end{bmatrix}$$

We seek to solve the normal equation:

$$A^T A x = A^T b$$

This can be done using the following program:

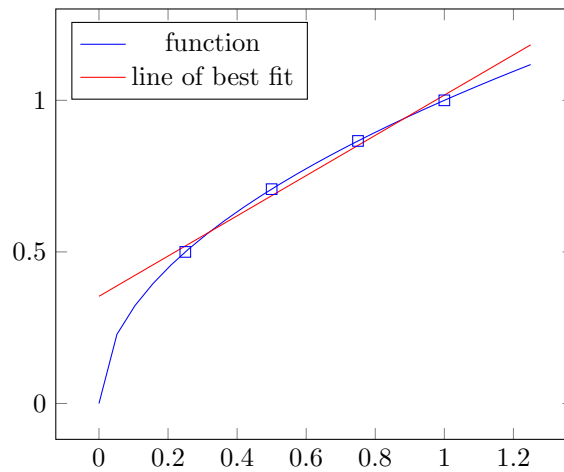
```
1 def transpose(A):
2     A_r, A_c = len(A), len(A[0])
3     A_T = [[A[i][j] for i in range(A_r)] for j in range(A_c)]
4     return A_T
5
6 def multiply(A, B):
7     A_r, A_c = len(A), len(A[0])
8     B_r, B_c = len(B), len(B[0])
9     AB = [[0 for _ in range(B_c)] for __ in range(A_r)]
10    for i in range(A_r):
11        for j in range(B_c):
12            for k in range(A_c):
13                AB[i][j] += A[i][k] * B[k][j]
14    return AB
15
16 def inverse(A):
17     A_ = A.copy()
18     A_rc = len(A_)
19     for i in range(A_rc):
20         A_[i].extend([int(i == j) for j in range(A_rc)])
21     for i in range(A_rc - 1, 0, -1):
22         if A_[i - 1][0] < A_[i][0]:
23             A_[i - 1], A_[i] = A_[i], A_[i - 1]
24     for i in range(A_rc):
25         for j in range(A_rc):
26             if j != i:
27                 tmp = A_[j][i] / A_[i][i]
28                 for k in range(2 * A_rc):
```

```

29         A_[j][k] -= A_[i][k] * tmp
30     for i in range(A_rc):
31         tmp = A_[i][i]
32         for j in range(2 * A_rc):
33             A_[i][j] = A_[i][j] / tmp
34     return [A_[i][A_rc:] for i in range(A_rc)]
35
36 m = 4
37
38 t = lambda i: 1/4 + 3/4 * (i - 1) / (m - 1)
39
40 A = [[1, t(i)] for i in range(1, m + 1)]
41 b = [[t(i) ** 0.5] for i in range(1, m + 1)]
42
43 x = multiply(inverse(multiply(transpose(A), A)), multiply(transpose(A), b))
44 print(x)

```

Plotting the function and the computed line of best fit:



Exercise 7

We solve the least squares problem where:

$$A = \begin{bmatrix} 1 & -1 & 0 & 0 \\ -1 & 0 & 1 & 0 \\ 1 & 0 & 0 & -1 \\ 0 & 0 & 1 & -1 \\ 0 & 1 & 0 & -1 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$

$$x = \begin{bmatrix} r_1 \\ r_2 \\ r_3 \\ r_4 \end{bmatrix}$$

$$b = \begin{bmatrix} 4 \\ 9 \\ 6 \\ 3 \\ 7 \\ 100 \end{bmatrix}$$

We seek to solve the normal equation:

$$A^T A x = A^T b$$

This can be done using the following program:

```

1 def transpose(A):
2     A_r, A_c = len(A), len(A[0])
3     A_T = [[A[i][j] for i in range(A_r)] for j in range(A_c)]
4     return A_T
5
6 def multiply(A, B):
7     A_r, A_c = len(A), len(A[0])
8     B_r, B_c = len(B), len(B[0])
9     AB = [[0 for _ in range(B_c)] for __ in range(A_r)]
10    for i in range(A_r):
11        for j in range(B_c):
12            for k in range(A_c):
13                AB[i][j] += A[i][k] * B[k][j]
14    return AB
15
16 def inverse(A):
17     A_ = A.copy()
18     A_rc = len(A_)
19     for i in range(A_rc):
20         A_[i].extend([int(i == j) for j in range(A_rc)])
21     for i in range(A_rc - 1, 0, -1):
22         if A_[i - 1][0] < A_[i][0]:
23             A_[i - 1], A_[i] = A_[i], A_[i - 1]
24     for i in range(A_rc):
25         for j in range(A_rc):
26             if j != i:
27                 tmp = A_[j][i] / A_[i][i]
28                 for k in range(2 * A_rc):

```

```

29         A_[j][k] -= A_[i][k] * tmp
30     for i in range(A_rc):
31         tmp = A_[i][i]
32         for j in range(2 * A_rc):
33             A_[i][j] = A_[i][j] / tmp
34     return [A_[i][A_rc:] for i in range(A_rc)]
35
36 A = [[1, -1, 0, 0], [-1, 0, 1, 0], [1, 0, 0, -1], [0, 0, 1, -1], [0, 1, 0, -1], [1, 1, 1, 1]]
37 b = [[4], [9], [6], [3], [7], [100]]
38
39 x = multiply(inverse(multiply(transpose(A), A)), multiply(transpose(A), b))
40 print(x)

```

This yields:

$$x = \begin{bmatrix} 25.25 \\ 24.625 \\ 29.125 \\ 21.0 \end{bmatrix}$$

We rank Team 3 first, Team 1 second, Team 2 third, and Team 4 fourth.